

PMP, End to End

Physical Memory Protection: what had to be fixed before a single test could run, how the entries get written, where the addresses come from, and how we know what we reached.

FIRST, THE THING PEOPLE GET WRONG

The solver does not choose the PMP settings

It is natural to assume the whole thing is solved: the model gives conditions, the solver finds values. That is true for **instruction operands**. It is not true for **machine state**, and PMP is machine state.

	INSTRUCTION OPERANDS	PMP ENTRIES
Who decides	The solver	We do — arithmetically
How	Path condition handed to Z3, which returns a satisfying value	An address is shifted and masked into the NAPOT encoding, in ordinary Rust
Result	<code>x11 = 0</code> because the branch needed a zero divisor	<code>li x31, 0x2000807f</code> — a constant, emitted into the preamble

Why not ask the solver for the PMP state too? Because it is not a search problem. There is no unknown to find — we already know which entry should deny which address; we just have to encode it. Asking a solver to rediscover an encoding we can compute directly costs solver time and makes the resulting test harder to read, with nothing gained. The solver is used where it earns its keep: deriving inputs that reach a named branch.

STEP 1

The foundation, and why nothing worked without it

PMP was the single hardest thing to make work, and for one reason. The model stores PMP configuration like this:

```
bitfield Pmpcfg_ent : bits(8) = {
  L : 7,           // locking
  A : 4 .. 3,      // address match type
  X : 2, W : 1, R : 0
}

register pmpcfg_n : vector(64, Pmpcfg_ent)
```

That is a **vector of bitfield structs** — not a vector of plain numbers. The symbolic engine's value representation had no case for struct-typed vector elements, so reading `pmpcfg_n[i]` symbolically crashed the engine outright. Every PMP test was blocked on that one gap.

What had to change, and where

REPOSITORY	FILE	CHANGE	WHY
isla	isla-lib/src/primop.rs	Give <code>vector_access</code> and <code>vector_update</code> a struct-typed branch that builds a per-field conditional	The blocker. Without it, any symbolic PMP access crashes
sail-riscv	pmp/pmp_regs.sail	<code>reset_pmp()</code> builds a fully concrete <code>Mk_Pmpcfg_ent(zeros())</code> per entry, instead of a <code>with</code> -update on the previous value	A <code>with</code> -update only concretises the fields it names. X/W/R stayed symbolic, and reading <code>.bits</code> in that half-concrete state hit the same engine gap from the other side
isla-testgen	src/target.rs	Add <code>pmp_deny()</code> , <code>pmp_allow_all()</code> , <code>run_in_supervisor()</code> to the target interface	The generator needs a way to be told what state to build
isla-testgen	src/generate_object_riscv.rs	Emit the preamble instructions that write the entries	Where the arithmetic actually happens
framework	python-isla/scenario_tests.py	Drive the flags, run both simulators, check the expectation	Turns a flag combination into a checked result

The model-side fix is worth understanding, because it looks unnecessary. The original `reset_pmp()` set only `A` and `L`, which is what the architecture requires at reset. But that leaves `X`, `W` and `R` holding whatever the register had before — symbolic, since the register has no default. Building the whole entry from zero is *equally correct architecturally* and avoids the problem entirely.

STEP 2

Setting up: how an entry actually gets written

The preamble is ordinary RISC-V assembly, emitted by the generator before the instruction under test. For a deny scenario it writes two entries:

```
/* deny at base, permit everything else */
li x31, (base >> 2) | 0x1ff      ← computed in Rust, emitted as a literal
csrw pmpaddr0, x31              ← entry 0 covers a region around base
li x31, -1
csrw pmpaddr1, x31              ← entry 1 covers everything
li x31, 0x1f98                  ← [0]=L|NAPOT|no perms, [1]=NAPOT|RWX
csrw pmpcfg0, x31
```

The three details that took thought

DETAIL	WHY IT IS THAT WAY
The denying entry must be entry 0	PMP is first-match-wins. Reversed, the catch-all matches first, nothing is ever denied, and the test passes while proving the opposite of what it claims

DETAIL	WHY IT IS THAT WAY
Entry 0 is locked (L=1)	A PMP entry only constrains Machine mode when locked. Locking lets the deny be tested without first dropping privilege
Entry 1 is not locked	It is the permissive catch-all; locking it would needlessly constrain M-mode everywhere else in the test

Allow-all is the other half, and it is not optional. `pmpCheck` denies every access matching no entry whenever privilege is not Machine — and the default configuration has PMP enabled with zero entries. So a test that leaves Machine mode cannot even *fetch its own next instruction* without an allow-all entry first. Every S-mode and U-mode scenario carries one.

STEP 3

Where the addresses come from

This is the part the model cannot tell us. Sail says “*an address inside the entry*”; only the platform configuration says which number that is.

VALUE	SOURCE	DERIVED OR CHOSEN
Base of main memory	<code>rv_ram_base</code> → <code>0x80000000</code>	Read from the platform config
The test's data address	<code>0x80020000</code>	Chosen inside RAM, clear of the code region
NAPOT encoding	<code>(addr >> 2) 0x1ff</code>	Computed — the encoding is arithmetic, not a choice
A non-matching address	<code>0x80028000</code>	Chosen outside the entry, for the negative control

Reading the NAPOT encoding

<code>address</code>	<code>0x80020000</code>	
<code>>> 2</code>	<code>0x20008000</code>	PMP addresses are in units of 4 bytes
<code> 0x1ff</code>	<code>0x200081ff</code>	trailing ones set the region size
		9 ones → $2^{(9+3)} = 4 \text{ KiB}$ region

The rule the model states is: the position of the last `0` dictates region size and alignment. Setting n trailing ones gives a region of 2^{n+3} bytes.

STEP 4

Executing, and checking the result

The generated ELF runs on the Golden Model and on Spike. What counts as a pass depends on whether the access was meant to be denied.

CASE	WHAT SHOULD HAPPEN	HOW IT IS CHECKED
Access permitted	The load completes	Ordinary final-state comparison — the loaded value matches
Access denied	An access fault, cause 5 or 7	<code>--expect-trap-cause</code> plus <code>--trap-is-pass</code> : the trap <i>is</i> the result

Why a denied access needs `--trap-is-pass`. Without it the handler advances past the faulting instruction and resumes, and then the ordinary final-state comparison runs — and fails by construction, because the faulting instruction never wrote its destination register, so no expected value can match. The architectural claim is “this access is denied”, and the trap is the evidence. There is no post-state to compare.

The controls, and what each one would catch

CONTROL	CHANGE FROM THE PASSING TEST	CATCHES
wrong cause	expect cause 7 instead of 5	A handler that notices a trap rather than <i>the</i> trap
no violation	deny a different address	A test that faults for some unrelated reason
S-mode default-deny	drop the allow-all	<code>--run-in-supervisor</code> silently not taking effect

Two of these scenarios passed vacuously during development and were caught only by building the case that had to fail. A privileged test that never reaches its trap looks exactly like one that traps correctly — both end in a pass.

STEP 5

Coverage: what we reached, and what the model says we missed

`pmp/` compiles to 226 instrumented spans. After running the suite we reconcile every one of them into hit, missed, or excluded.

The dimensions, all declared in the Sail

DIMENSION	DECLARED IN	CASES
Match mode	enum PmpAddrMatchType	4
Match outcome	enum pmpAddrMatch	3
Access kind	pmpCheckRWX arms	19
Permissions, lock	bitfield Pmpcfg_ent	L, A, X, W, R
Privilege	enum Privilege	3 relevant
Grain, entry count	config memory.pmp.*	config

The space does not explode, because `pmpCheck` composes three independent functions. Address matching depends only on the address; the permission check only on permissions and access kind; the privilege override only on privilege and the lock bit. They do not interact, so the cases add rather than multiply: roughly **ten** address cases plus **thirty-two** permission cases plus **six** override cases — about fifty, not the eight thousand a full cross-product would give.

What the enumeration finds that a hand-written list did not

`PMP_PartialMatch` is a declared outcome that no current scenario reaches. It is an access that *partially overlaps* an entry, which the model treats as an immediate access fault. Nobody thought to write a straddling access by

hand. Enumerating from the enum finds it on the first pass — which is the argument for deriving the case space rather than curating it.

pmpCheck 's unmatched arm is not reachable today. Its final line — `if priv == Machine then None() else Some(accessFault...)` — is the only place privilege alone decides. Both current flags guarantee a match: allow-all installs a catch-all, deny installs a denying entry *plus* a permitting one. Reaching it needs a bounded-range option (`--pmp-allow-range lo-hi`) that leaves the data address unmatched while keeping the harness's own code fetchable.

Three arms are excluded, not missed. `pmpCheckRWX` 's three `internal_error` arms are unreachable by construction — they exist to catch a malformed access the decoder cannot produce. They are recorded as exclusions with that reason, so they never appear as a coverage gap.

SUMMARY

The whole thing in one table

QUESTION	ANSWER
What blocked it?	Struct-typed vector elements in the symbolic engine
Who picks the entries?	We do, arithmetically. Not the solver
Who picks the addresses?	The platform config's memory map, plus arithmetic for the NAPOT encoding
Where does the case space come from?	The model's enums, bitfield and match arms — all declared
How big is it?	~50 cases, because the check factorises into three independent parts
How do we know we are done?	226 spans in <code>pmp/</code> , each reconciled to hit, missed, or excluded-with-reason

Encodings and comments read from `src/generate_object_riscv.rs`; the case space from `model/pmp/pmp_control.sail` and `pmp_regs.sail`; scenarios from `python-isla/scenario_tests.py`.